AGENTIC AI WITH JAVA

Study Program

LLMs Deep Dive + Java Setup

Industry-Ready Comprehensive Study Guide

Session 1	Session 2	Session 3	Session 4
LLM Theory	Java + LLMs Setup	Hugging Face	Spring AI

Understanding Large Language Models

Session 1: Understanding Large Language Models

Large Language Models (LLMs) represent the foundational technology powering everything from ChatGPT to GitHub Copilot. As an Agentic AI developer, deeply understanding how LLMs work — not just as black boxes but as engineering systems — is non-negotiable. This session builds that foundation.

1.1 Open Source vs Closed Source LLMs

The LLM landscape is broadly divided into two categories based on how models are distributed and accessed.

Closed Source LLMs

Closed source models are proprietary systems developed and maintained by private companies. The weights (the actual parameters learned during training) are not publicly released. You access them exclusively through APIs.

Model	Provider & Key Characteristics
GPT-4o / GPT-4	OpenAI. Industry gold standard. Excellent reasoning, code, multimodal. Pay-per-token via API.
Gemini 1.5 Pro	Google DeepMind. Massive 1M token context window. Deep Google ecosystem integration.
Claude 3.5 Sonnet	Anthropic. Strong safety focus, excellent for instruction-following, 200K context.
Command R+	Cohere. Optimized for enterprise RAG and retrieval augmented generation.

Open Source LLMs

Open source models have their weights publicly released, meaning you can download and run them on your own hardware. This unlocks fine-tuning, local deployment, and no per-token cost.

Model	Developer & Key Characteristics
Llama 3.1 / 3.2	Meta. Available in 8B, 70B, 405B sizes. Most popular open-source family for production.
Mistral 7B / Mixtral 8x7B	Mistral AI. Efficient, high-quality models. Mixtral uses Mixture-of-Experts (MoE) architecture.

Falcon	Technology Innovation Institute. Strong multilingual capabilities.
Phi-3	Microsoft. Small but surprisingly capable models optimized for edge devices.

Dimension	Open Source	Closed Source
Cost	Free weights; pay for compute	Pay per token (API pricing)
Privacy	Full data control, runs locally	Data sent to provider servers
Customization	Full fine-tuning possible	Limited (prompt only / fine-tune APIs)
Maintenance	You manage updates	Provider handles everything
Performance	Smaller models may lag	Usually state-of-the-art
Best For	Privacy-sensitive, offline apps	Rapid prototyping, best quality

Industry Insight

In production Agentic AI systems, a hybrid approach is common: closed-source models for tasks requiring maximum intelligence (reasoning, planning), open-source models for high-volume, cost-sensitive subtasks (classification, extraction).

1.2 Tokens — The Atomic Unit of LLMs

Every LLM processes and generates text in units called tokens. Understanding tokens is critical because API pricing, context limits, speed, and behaviour all revolve around token counts.

What Is a Token?

A token is not a word or a character — it is a sub-word unit determined by the model's tokenizer. The most common tokenization algorithm used in modern LLMs is Byte Pair Encoding (BPE).

Rule of Thumb

1 token ≈ 4 characters in English. 1000 tokens ≈ 750 words. Non-English languages and code can be less efficient — some languages use 2–4x more tokens per word than English.

How Tokenization Works

1. The tokenizer starts with individual characters as the vocabulary.
2. It repeatedly merges the most frequently co-occurring pair of symbols.
3. This continues until the vocabulary reaches a target size (usually 32K–100K tokens).

Example — how GPT tokenizes text:

```
Input:  "unhappiness"
```

```
Tokens: ["un", "happ", "iness"]   → 3 tokens

Input:  "the"
Tokens: ["the"]                   → 1 token (common word = single token)

Input:  "supercalifragilistic"
Tokens: ["super", "cal", "if", "rag", "ilis", "tic"]  → 6 tokens
```

Why Tokens Matter in Practice

- API Cost: OpenAI charges per 1M tokens (input + output). GPT-4o at $5/1M input tokens.
- Speed: More tokens = slower generation. Critical for real-time agentic applications.
- Context Limits: Models have a fixed context window measured in tokens.
- Prompt Engineering: Verbose prompts waste tokens; concise prompts save money.

> **Pro Tip**
> *Use the OpenAI Tokenizer playground at platform.openai.com/tokenizer to visualize exactly how any text is tokenized. This is essential for estimating API costs before deploying your Java application.*

1.3 Context Windows — Understanding Limits and Implications

The context window (also called context length) is the maximum number of tokens a model can process in a single request — both the input prompt AND the generated output combined.

Model	Context Window
GPT-4o	128,000 tokens (~96,000 words)
Claude 3.5 Sonnet	200,000 tokens (~150,000 words)
Gemini 1.5 Pro	1,000,000 tokens (~750,000 words)
Llama 3.1 70B	128,000 tokens
Mistral 7B	32,000 tokens

What Lives in the Context Window?

- System prompt (instructions to the model)
- Full conversation history (all prior messages)
- Retrieved documents (in RAG applications)
- Tool call results (in agentic workflows)
- The model's response being generated

Key Implications for Agentic AI Development

Context windows have profound implications for how you architect AI applications:

Long Conversations Problem

In a chatbot with no memory management, every new message appends to the full history. After 50 messages, you might hit the context limit. Solution: use sliding window, summarization, or vector store memory.

Cost vs Context Trade-off

Larger context = more tokens processed = higher cost per call. A 100-page document in context might cost $0.50 per query with GPT-4o. Design your system to only include relevant context (RAG pattern).

Lost in the Middle Problem

Research shows LLMs are best at processing information at the start and end of their context. Information buried in the middle of a very long context may be 'forgotten'. Keep critical instructions at the start.

1.4 Sampling Strategies — Controlling Model Output

When an LLM generates the next token, it doesn't just pick the single most probable token every time (that would produce dull, repetitive text). Instead, it samples from a probability distribution. Several hyperparameters control this sampling behaviour.

Temperature

Temperature controls the randomness/creativity of output. It scales the logits (raw model outputs) before the softmax function converts them to probabilities.

Temperature	Behaviour	Best Use Case
0.0	Deterministic — always picks highest probability token	Code generation, factual Q&A, JSON extraction
0.1 – 0.4	Very focused, mostly consistent	Summarization, classification, structured tasks
0.7 – 0.9	Balanced creativity and coherence	General chat, content writing, brainstorming
1.0+	Highly creative, unpredictable	Creative writing, poetry, idea generation

Top-P (Nucleus Sampling)

Top-P sampling selects from the smallest set of tokens whose cumulative probability exceeds P. Rather than considering all tokens or just the top-K, it dynamically adjusts the candidate set based on confidence.

Example — Top-P = 0.9

If the model assigns high probability to a few tokens (confident), only those few are considered. If it's uncertain and spreads probability across many tokens, a larger set is considered. This makes top-p adaptive in a way that top-k is not.

Top-K

Top-K sampling restricts selection to the K most probable next tokens and redistributes probability among only those K candidates. For example, with top-k=50, only the 50 most probable tokens are considered regardless of their individual probabilities.

Practical Guidance for Java Applications

- Temperature: Set to 0 for deterministic tool calls in agentic workflows.
- Temperature: Set to 0.7 for natural conversational responses.
- Top-P: Default 1.0 works well; lower to 0.9 for more focused outputs.
- Avoid changing both Temperature and Top-P simultaneously — it's hard to reason about the combined effect.

Pro Tip

In Spring AI, you set these via ChatOptions: ChatOptionsBuilder.builder().withTemperature(0.7f).withTopP(0.95f).build(). Always document what temperature you used and why, as it affects reproducibility of your AI application.

1.5 Hallucinations — Understanding and Mitigating AI Confabulation

Hallucination is the phenomenon where an LLM confidently generates text that is factually incorrect, fabricated, or not grounded in its training data or provided context. It is one of the most critical challenges in production AI systems.

Why Do Hallucinations Occur?

LLMs are fundamentally pattern-matching and text-completion systems trained to produce plausible-sounding text. They do not 'know' facts in the way humans do — they predict what text is statistically likely to follow a given input.

- Training data gaps: If the model was not trained on accurate information about a topic, it will generate plausible-sounding but incorrect text.
- Knowledge cutoff: Models have a training cutoff date. Events or information after that date are unknown to the model.
- Conflicting training data: If the training corpus contains contradictory information, the model may blend them unpredictably.
- Overly helpful design: Models are trained to always provide an answer, even when the honest answer is 'I don't know'.

- Lack of grounding: Without access to verified sources, the model has no mechanism to verify its outputs.

Types of Hallucinations

Type	Description & Example
Factual Hallucination	Model states an incorrect fact. E.g., 'The Eiffel Tower was built in 1887' (it was 1889).
Citation Hallucination	Model fabricates academic papers, books, or URLs that do not exist.
Entity Confusion	Model conflates two similar entities. E.g., confusing two different politicians with similar names.
Reasoning Hallucination	Model produces a plausible-sounding but logically flawed chain of reasoning.
Code Hallucination	Model generates syntactically valid code that uses non-existent APIs or functions.

Mitigation Strategies

1. Retrieval Augmented Generation (RAG): Ground the model by providing verified source documents in the context. Instruct the model to answer only from provided context.
2. System Prompt Constraints: Explicitly instruct the model to say 'I don't know' when uncertain rather than guessing.
3. Temperature = 0 for factual tasks: Lower temperatures reduce creative confabulation.
4. Response Verification: Use a second LLM call to fact-check the first response against provided sources.
5. Structured Output + Validation: Force the model to respond in JSON; validate the schema programmatically.
6. Human-in-the-Loop: For high-stakes decisions in agentic workflows, require human approval before executing actions.

Key Principle for Agentic AI

In agentic systems where the AI takes real-world actions (calling APIs, writing files, sending emails), hallucinations can cause real harm. Always build verification layers and prefer retrieval-grounded approaches over relying on parametric model memory.

1.6 Cost and Latency Trade-offs

Production AI applications must balance three competing objectives: quality of output, cost per request, and response latency. Understanding these trade-offs is essential for architecting systems that are both effective and economically viable.

Cost Dimensions

- Input token cost: What you pay per token in the prompt (usually cheaper than output).
- Output token cost: What you pay per token the model generates (typically 3–5x more expensive than input).
- Compute cost (self-hosted): GPU rental cost if running open-source models on your own infrastructure.

Model	Input (per 1M tokens)	Output (per 1M tokens)
GPT-4o	$5.00	$15.00
GPT-4o mini	$0.15	$0.60
Claude 3.5 Sonnet	$3.00	$15.00
Claude 3 Haiku	$0.25	$1.25
Llama 3.1 8B (self-hosted)	~$0.05	~$0.05

Latency Factors

- Model size: Larger models (70B parameters) are slower than smaller ones (7B).
- Context length: Processing more input tokens takes longer (attention is quadratic in sequence length).
- Output length: More tokens to generate means more time.
- Infrastructure: API calls add network latency; self-hosted adds cold-start time.

Optimization Strategies

Model Routing

Use a cheap/fast model (GPT-4o mini, Haiku) for simple subtasks and only escalate to expensive models when necessary. In Spring AI, this is implemented via ModelSelector or conditional routing logic.

Caching

Cache responses for identical or near-identical prompts using semantic caching (e.g., Redis with vector similarity). OpenAI and Anthropic offer automatic prompt caching that discounts repeated context up to 90%.

Streaming

For user-facing applications, use streaming responses (Server-Sent Events). The first token appears in ~500ms even if the full response takes 10 seconds. Spring AI fully supports streaming via Flux<ChatResponse>.

Pro Tip

Build a cost estimator before deploying. Take a sample of 100 real prompts, count average tokens, and multiply by your target daily request volume. A system doing 10,000 requests/day at 500 tokens average with GPT-4o costs $750/day in output alone.

Key Terms — Session 1 Quick Reference

Term	Definition
Token	Sub-word unit used by LLM tokenizers. ~4 chars in English. Basis for API pricing.
Context Window	Max tokens a model can process (input + output) in one request.
Temperature	Controls output randomness. 0 = deterministic, 1.0+ = creative.
Top-P	Nucleus sampling — sample from tokens covering top P% of cumulative probability.
Top-K	Restrict sampling to K highest-probability tokens only.
Hallucination	Model generating confident but factually incorrect or fabricated content.
RAG	Retrieval Augmented Generation — grounding model answers in retrieved documents.
Logits	Raw unnormalized scores output by the model before softmax converts to probabilities.

SESSION 2

Java + LLMs Setup

Session 2: Java + LLMs Setup

This session transitions from theory to hands-on development. By the end, you will have a working Java application that communicates with an LLM. Understanding the right abstraction layer to use — REST API, SDK, or Framework — is the first decision every Java AI developer must make.

2.1 API vs SDK vs Framework — Understanding the Abstraction Layers

When integrating an LLM into a Java application, you can work at three different levels of abstraction. Each has distinct trade-offs.

Layer	What It Is	When to Use
Raw REST API	Direct HTTP calls to provider endpoints using curl/HttpClient/RestTemplate	Learning internals, maximum control, polyglot systems
SDK	Provider-supplied library that wraps the API with typed Java classes	Standard integrations; balances control and convenience
Framework (Spring AI)	Opinionated abstraction over multiple providers with Spring Boot auto-config	Production apps; provider portability; Spring ecosystem integration

The Layered Mental Model

Think of It This Way

Raw API = assembling furniture with raw materials. SDK = IKEA kit with instructions. Framework = calling a professional furniture service. Each abstraction removes complexity but also reduces control.

2.2 OpenAI REST API — Calling via curl and Postman

Before writing any Java code, every AI developer should understand the underlying HTTP API. The OpenAI Chat Completions API is the most widely adopted interface, and many other providers (Anthropic, Groq, Together AI) have adopted compatible formats.

Core Endpoint

POST https://api.openai.com/v1/chat/completions

Required Headers

```
Authorization: Bearer sk-YOUR_API_KEY_HERE
Content-Type: application/json
```

Request Body Structure

```
{
  "model": "gpt-4o-mini",
  "messages": [
    {
      "role": "system",
      "content": "You are a helpful Java programming assistant."
    },
    {
      "role": "user",
      "content": "Explain what a context window is in 2 sentences."
    }
  ],
  "temperature": 0.7,
  "max_tokens": 500
}
```

curl Example (Terminal)

```
curl https://api.openai.com/v1/chat/completions \
  -H "Authorization: Bearer $OPENAI_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{
    "model": "gpt-4o-mini",
    "messages": [{"role": "user", "content": "Hello World"}],
    "temperature": 0.7
  }'
```

Understanding the Response

```
{
  "id": "chatcmpl-abc123",
  "object": "chat.completion",
  "model": "gpt-4o-mini",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": "Hello! How can I help you today?"
      },
      "finish_reason": "stop"
    }
  ],
  "usage": {
    "prompt_tokens": 12,
    "completion_tokens": 9,
    "total_tokens": 21
```

```
    }
}
```

finish_reason Value	Meaning
stop	Model finished naturally — reached end of response
length	Hit max_tokens limit — response was truncated
tool_calls	Model wants to call a function/tool
content_filter	Response blocked by safety filter

Message Roles Explained

Role	Purpose & Usage
system	Sets the AI's persona and constraints. Applied once at the start. Never overridden by user messages.
user	The human's input message.
assistant	The AI's prior responses. Include in conversation history for multi-turn.
tool	Results returned from tool/function calls. Used in agentic workflows.

> **Pro Tip**
> *Test every API call in Postman before implementing in Java. Postman's 'Code snippet' feature (top right of request panel) can auto-generate the Java HttpClient code for your request.*

2.3 OpenAI Java SDK

The official openai-java SDK provides type-safe access to all OpenAI APIs. It handles HTTP, retries, authentication, streaming, and response parsing.

Maven Dependency

```
<dependency>
    <groupId>com.openai</groupId>
    <artifactId>openai-java</artifactId>
    <version>0.8.0</version>
</dependency>
```

Gradle Dependency

```
implementation 'com.openai:openai-java:0.8.0'
```

Simple Chat Completion

```
import com.openai.client.OpenAIClient;
```

```
import com.openai.client.okhttp.OpenAIOkHttpClient;
import com.openai.models.*;

OpenAIClient client = OpenAIOkHttpClient.builder()
    .apiKey(System.getenv("OPENAI_API_KEY"))
    .build();

ChatCompletion completion = client.chat().completions().create(
    ChatCompletionCreateParams.builder()
        .model(ChatModel.GPT_4O_MINI)
        .addUserMessage("What are tokens in the context of LLMs?")
        .temperature(0.7)
        .maxTokens(500)
        .build()
);

String response = completion.choices().get(0).message().content().get();
System.out.println(response);
```

Multi-turn Conversation with SDK

```
List<ChatCompletionMessageParam> messages = new ArrayList<>();
messages.add(ChatCompletionMessageParam.ofSystem(
    "You are an expert Java developer and AI educator."
));
messages.add(ChatCompletionMessageParam.ofUser(
    "Explain Spring AI in one paragraph."
));

ChatCompletion response1 = client.chat().completions().create(
    ChatCompletionCreateParams.builder()
        .model(ChatModel.GPT_4O_MINI)
        .messages(messages)
        .build()
);

String assistantReply = response1.choices().get(0).message().content().get();
messages.add(ChatCompletionMessageParam.ofAssistant(assistantReply));
messages.add(ChatCompletionMessageParam.ofUser("Can you give a code example?"));

ChatCompletion response2 = client.chat().completions().create(
    ChatCompletionCreateParams.builder()
        .model(ChatModel.GPT_4O_MINI)
        .messages(messages)
        .build()
);
```

Streaming Response

```
client.chat().completions().createStreaming(
    ChatCompletionCreateParams.builder()
        .model(ChatModel.GPT_4O_MINI)
        .addUserMessage("Write a haiku about Java")
```

```
        .build()
).subscribe(chunk -> {
    chunk.choices().stream()
        .map(c -> c.delta().content())
        .flatMap(Optional::stream)
        .forEach(System.out::print);
});
```

2.4 Overview of Other Java AI SDKs

Beyond the OpenAI SDK, the Java AI ecosystem includes several other options worth knowing:

SDK / Library	Description & Key Features
LangChain4j	Most popular Java AI framework. Chains, agents, RAG, memory, tool use. Provider-agnostic. Similar to Python's LangChain.
Spring AI	Spring Boot-native. Auto-configuration, ChatClient abstraction, RAG support. Best for Spring applications.
Anthropic Java SDK	Official SDK for Claude models. Comparable to OpenAI SDK in structure.
Google Vertex AI Java	Official SDK for Gemini and Vertex AI endpoints. Integrates with GCP ecosystem.
Ollama4j	Java client for the Ollama API — run local LLMs (Llama, Mistral) on developer machines.

2.5 Building Your First AI App in Java

Now we bring everything together. The following is a complete, runnable Java console application that calls the OpenAI API and handles a simple conversation.

Project Structure

```
ai-hello-world/
├── pom.xml
└── src/main/java/com/agenticai/
    ├── App.java
    └── ConversationService.java
```

pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project>
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.agenticai</groupId>
  <artifactId>ai-hello-world</artifactId>
  <version>1.0.0</version>

  <dependencies>
```

```
    <dependency>
      <groupId>com.openai</groupId>
      <artifactId>openai-java</artifactId>
      <version>0.8.0</version>
    </dependency>
  </dependencies>
</project>
```

App.java — Main Entry Point

```
package com.agenticai;

import com.openai.client.OpenAIClient;
import com.openai.client.okhttp.OpenAIOkHttpClient;

public class App {
    public static void main(String[] args) {
        String apiKey = System.getenv("OPENAI_API_KEY");
        if (apiKey == null || apiKey.isEmpty()) {
            throw new RuntimeException("OPENAI_API_KEY env variable not set");
        }

        OpenAIClient client = OpenAIOkHttpClient.builder()
            .apiKey(apiKey)
            .build();

        ConversationService service = new ConversationService(client);

        System.out.println("=== Agentic AI Hello World ===");
        String response = service.chat("Hello! I'm a Java developer learning about
LLMs.");
        System.out.println("AI: " + response);

        String response2 = service.chat("What should I learn first about Spring
AI?");
        System.out.println("AI: " + response2);
    }
}
```

ConversationService.java

```
package com.agenticai;

import com.openai.client.OpenAIClient;
import com.openai.models.*;
import java.util.ArrayList;
import java.util.List;

public class ConversationService {
    private final OpenAIClient client;
    private final List<ChatCompletionMessageParam> history = new ArrayList<>();

    public ConversationService(OpenAIClient client) {
```

```
        this.client = client;
        history.add(ChatCompletionMessageParam.ofSystem(
            "You are an expert Java and AI developer. " +
            "Provide concise, practical answers for a developer learning Agentic
AI."
        ));
    }

    public String chat(String userMessage) {
        history.add(ChatCompletionMessageParam.ofUser(userMessage));

        ChatCompletion completion = client.chat().completions().create(
            ChatCompletionCreateParams.builder()
                .model(ChatModel.GPT_4O_MINI)
                .messages(history)
                .temperature(0.7)
                .maxTokens(500)
                .build()
        );

        String assistantReply =
completion.choices().get(0).message().content().get();
        history.add(ChatCompletionMessageParam.ofAssistant(assistantReply));
        return assistantReply;
    }
}
```

Pro Tip

Before writing any code, always test your API key with a single curl call. This isolates authentication issues from code issues. Export your key: export OPENAI_API_KEY=sk-... and then run the curl example from section 2.2.

SESSION 3

Hugging Face Integration

Session 3: Hugging Face Integration

Hugging Face is the GitHub of machine learning — the central hub where researchers and companies publish models, datasets, and demos. With over 800,000 public models (as of 2025), it is where you find open-source alternatives to every closed-source model. As a Java Agentic AI developer, knowing how to integrate Hugging Face models into your applications opens an enormous library of capabilities.

3.1 Hugging Face Platform Overview

Hugging Face offers several interconnected products:

Product	Description & Use Case
Model Hub	Repository of 800K+ pre-trained models. Browse, test, and download models for NLP, vision, speech, and more.
Datasets Hub	Largest open dataset repository. Used for training and evaluation.
Spaces	Hosted demos of ML applications (Gradio/Streamlit). Try models without any setup.
Inference API	REST API to run inference on hosted models. No GPU required — perfect for Java integration.
Inference Endpoints	Dedicated, scalable deployment of any HF model on cloud infrastructure. For production workloads.
Transformers Library	Core Python library. Not directly used from Java, but important to understand.

Model Cards — Reading Them Effectively

Every model on Hugging Face has a Model Card — a documentation page describing what the model does, how it was trained, its limitations, and how to use it. As a Java developer integrating these models, you must read model cards to understand:

- Task type (text-generation, fill-mask, token-classification, etc.)
- Input format requirements (special tokens, prompt templates)
- Output format (what the JSON response contains)
- License (MIT, Apache 2.0, Llama Community License, etc.)
- Model size and expected inference time

3.2 Inference API — Calling HF Models from Java

The Hugging Face Inference API is a free (rate-limited) REST API that lets you run inference on any supported model without managing GPU infrastructure. The endpoint format is:

POST https://api-inference.huggingface.co/models/{model-id}

Supported Task Types and Their Payloads

Task	Model Example	Input Format
Text Generation	mistralai/Mistral-7B-v0.1	{"inputs": "Your prompt here"}
Text Classification	distilbert-base-uncased-finetuned-sst-2-english	{"inputs": "I love this!"}
Named Entity Recog.	dslim/bert-base-NER	{"inputs": "Paris is in France"}
Question Answering	deepset/roberta-base-squad2	{"inputs": {"question": "...", "context": "..."}}
Summarization	facebook/bart-large-cnn	{"inputs": "Long text to summarize..."}
Feature Extraction	sentence-transformers/all-MiniLM-L6-v2	{"inputs": "Text to embed"}

3.3 Authentication — Tokens and API Keys

Free access to the Inference API is possible for public models without authentication, but rate limits are very tight. For any real development work, create a free account and generate an access token.

Generating a HF Token

7. Go to huggingface.co → Sign up for a free account
8. Navigate to Settings → Access Tokens
9. Click 'New Token' → choose 'Read' role for inference
10. Copy the token — starts with 'hf_'

> **Token Roles**
> Read tokens: Can call inference on public and gated models you've been granted access to. Write tokens: Can push models and datasets to your account. For Inference API usage, a Read token is sufficient.

Authentication in API Calls

```
Authorization: Bearer hf_YOUR_TOKEN_HERE
```

3.4 Java REST Integration with Hugging Face APIs

Java's built-in HttpClient (Java 11+) is sufficient for calling the HF Inference API. The following examples demonstrate calling different task types.

Utility Class — HuggingFaceClient.java

```
package com.agenticai.hf;

import java.net.URI;
import java.net.http.*;
import java.net.http.HttpRequest.BodyPublishers;
import java.net.http.HttpResponse.BodyHandlers;

public class HuggingFaceClient {

    private static final String BASE_URL = "https://api-
inference.huggingface.co/models/";
    private final String token;
    private final HttpClient httpClient;

    public HuggingFaceClient(String token) {
        this.token = token;
        this.httpClient = HttpClient.newHttpClient();
    }

    public String infer(String modelId, String jsonBody) throws Exception {
        HttpRequest request = HttpRequest.newBuilder()
            .uri(URI.create(BASE_URL + modelId))
            .header("Authorization", "Bearer " + token)
            .header("Content-Type", "application/json")
            .POST(BodyPublishers.ofString(jsonBody))
            .build();

        HttpResponse<String> response = httpClient.send(request,
BodyHandlers.ofString());

        if (response.statusCode() == 503) {
            throw new RuntimeException("Model is loading — retry in 20 seconds");
        }
        if (response.statusCode() != 200) {
            throw new RuntimeException("HF API Error: " + response.statusCode() + "
" + response.body());
        }
        return response.body();
    }
}
```

Sentiment Analysis Example

```
HuggingFaceClient client = new HuggingFaceClient(System.getenv("HF_TOKEN"));

String payload = """
    {"inputs": "I absolutely love building AI applications in Java!"}
    """;
```

```
String result = client.infer(
    "distilbert-base-uncased-finetuned-sst-2-english",
    payload
);
System.out.println(result);
// Output: [[{"label":"POSITIVE","score":0.9998}]]
```

Text Generation Example

```
String payload = """
    {
      "inputs": "The key difference between RAG and fine-tuning is",
      "parameters": {
        "max_new_tokens": 200,
        "temperature": 0.7,
        "return_full_text": false
      }
    }
    """;

String result = client.infer("mistralai/Mistral-7B-Instruct-v0.3", payload);
System.out.println(result);
```

Embeddings / Feature Extraction

```
String payload = """
    {"inputs": "Agentic AI systems use LLMs to take autonomous actions"}
    """;

String embeddings = client.infer(
    "sentence-transformers/all-MiniLM-L6-v2",
    payload
);
// Returns a JSON array of floats representing the 384-dimensional embedding vector
```

Handling Model Loading (HTTP 503)

Public models on the Inference API are not always hot-loaded. The first call after a period of inactivity returns HTTP 503 with a JSON body indicating the estimated loading time. Your Java code must handle this gracefully:

```
public String inferWithRetry(String modelId, String body, int maxRetries) throws
Exception {
    for (int attempt = 1; attempt <= maxRetries; attempt++) {
        try {
            return infer(modelId, body);
        } catch (RuntimeException e) {
            if (e.getMessage().contains("loading") && attempt < maxRetries) {
                System.out.println("Model loading, waiting 20 seconds... (attempt "
+ attempt + ")");
                Thread.sleep(20_000);
            } else {
                throw e;
```

```
            }
        }
    }
    throw new RuntimeException("Max retries exceeded");
}
```

3.5 Spring AI Hugging Face Provider Setup

Spring AI integrates with Hugging Face's Text Generation Inference (TGI) and Inference API endpoints through its ChatModel abstraction. This allows you to swap between OpenAI and HF models with minimal code changes.

Maven Dependency

```
<dependency>
    <groupId>org.springframework.ai</groupId>
    <artifactId>spring-ai-huggingface-spring-boot-starter</artifactId>
    <version>1.0.0</version>
</dependency>
```

application.yml

```
spring:
  ai:
    huggingface:
      chat:
        api-key: ${HF_TOKEN}
        model: mistralai/Mistral-7B-Instruct-v0.3
        options:
          temperature: 0.7
          max-new-tokens: 500
```

Using HuggingFace ChatModel in a Service

```
@Service
public class HFChatService {

    private final ChatModel chatModel;

    public HFChatService(ChatModel chatModel) {
        this.chatModel = chatModel;
    }

    public String askQuestion(String question) {
        return chatModel.call(
            new Prompt(
                List.of(
                    new SystemMessage("You are a helpful Java AI assistant."),
                    new UserMessage(question)
                )
            )
```

```
            ).getResult().getOutput().getContent();
        }
}
```

Pro Tip

Create a free Hugging Face account and spend 20 minutes in the Model Hub. Use the 'Inference API' widget on any model page to test it directly in the browser — no code needed. Start with 'distilbert-base-uncased-finetuned-sst-2-english' for sentiment and 'facebook/bart-large-cnn' for summarization.

Key Terms — Session 3 Quick Reference

Term	Definition
Model Hub	HuggingFace's repository of 800K+ public ML models.
Inference API	HF's hosted REST endpoint to run any model without managing infrastructure.
Model Card	Documentation page for a model: task, training data, limitations, and usage.
Feature Extraction	Generating embedding vectors from text using encoder models.
TGI	Text Generation Inference — HF's optimized server for deploying LLMs at scale.
HTTP 503	Model is cold-loading on HF. Expected on first call after inactivity; retry after 20s.
Gated Model	Models requiring account verification before download/inference (e.g., Llama 3).

SESSION 4
Spring AI Foundation

Session 4: Spring AI Foundation

Spring AI is the official Spring ecosystem framework for building AI-powered Java applications. It provides a unified, provider-agnostic abstraction over multiple LLM providers, vector stores, embedding models, and tool execution frameworks. If you know Spring Boot, Spring AI will feel immediately familiar.

4.1 Spring AI Architecture and Design Philosophy

Spring AI was built around four core design principles:

11. Portability: Write code once, switch between OpenAI, Anthropic, HuggingFace, or Ollama by changing a configuration property.
12. Spring-native: Follows Spring Boot conventions (auto-configuration, dependency injection, properties-based config). No boilerplate.
13. Production-ready: Includes observability, retry, token counting, structured output, streaming, and RAG out of the box.
14. Composability: Provides building blocks (ChatClient, EmbeddingModel, VectorStore) that compose into complex AI pipelines.

Core Abstractions

Abstraction	Role & Description
ChatModel	Low-level interface for sending prompts and receiving responses. One implementation per provider.
ChatClient	High-level fluent API. Handles system prompts, advisors, memory, tools in a builder pattern.
EmbeddingModel	Converts text to vector embeddings. Used for RAG and semantic search.
VectorStore	Stores and retrieves embedding vectors. Integrates with pgvector, Chroma, Pinecone, etc.
DocumentReader	Reads documents from various sources (PDF, web, Word) for RAG ingestion pipelines.
Advisor	Middleware for ChatClient. Used to implement memory, logging, RAG retrieval, guardrails.
PromptTemplate	Template engine for constructing prompts with variable substitution.
OutputConverter	Converts model string output to typed Java objects (POJOs, Lists, Maps).

4.2 Supported Providers

Spring AI supports the following AI providers through auto-configured starters:

Provider	Models Available	Starter Artifact
OpenAI	GPT-4o, GPT-4o-mini, o1, o3	spring-ai-openai-spring-boot-starter
Anthropic	Claude 3.5 Sonnet/Haiku, Claude 3 Opus	spring-ai-anthropic-spring-boot-starter
Ollama	Llama 3, Mistral, Phi-3, Gemma2 (local)	spring-ai-ollama-spring-boot-starter
HuggingFace	Any TGI-compatible model	spring-ai-huggingface-spring-boot-starter
Google Vertex	Gemini 1.5 Pro, Gemini Flash	spring-ai-vertex-ai-gemini-spring-boot-starter
Azure OpenAI	GPT-4 via Azure endpoints	spring-ai-azure-openai-spring-boot-starter
Mistral AI	Mistral Large, Mixtral 8x22B	spring-ai-mistral-ai-spring-boot-starter

4.3 Auto-Configuration with Spring Boot

Spring AI leverages Spring Boot's auto-configuration system. When you add a starter dependency to your pom.xml and provide the required API key in application.yml, Spring AI automatically creates and configures all the necessary beans.

How Auto-Configuration Works

- Spring Boot detects the starter on the classpath via @ConditionalOnClass.
- Spring AI's auto-configuration class creates a ChatModel bean using properties from application.yml.
- You inject ChatModel or ChatClient into your @Service class via constructor injection.
- To switch providers, change the Maven dependency and update application.yml. Your service code is unchanged.

The Power of Provider Portability

In production, you might use OpenAI for development (best quality), switch to Azure OpenAI for enterprise compliance, and add Ollama for a local offline mode — all without changing a single line of business logic. Only configuration changes.

4.4 Complete Project Setup

Spring Initializr Setup

Visit start.spring.io and configure:

Setting	Value
Project	Maven Project
Language	Java
Spring Boot Version	3.4.x or latest stable
Group	com.agenticai
Artifact	spring-ai-demo
Java Version	21
Dependencies	Spring Web, Spring AI OpenAI

Complete pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.4.1</version>
  </parent>

  <groupId>com.agenticai</groupId>
  <artifactId>spring-ai-demo</artifactId>
  <version>1.0.0</version>

  <properties>
    <java.version>21</java.version>
    <spring-ai.version>1.0.0</spring-ai.version>
  </properties>

  <dependencyManagement>
    <dependencies>
      <dependency>
        <groupId>org.springframework.ai</groupId>
        <artifactId>spring-ai-bom</artifactId>
        <version>${spring-ai.version}</version>
        <type>pom</type>
        <scope>import</scope>
      </dependency>
    </dependencies>
  </dependencyManagement>

  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
```

```
    </dependency>
    <dependency>
      <groupId>org.springframework.ai</groupId>
      <artifactId>spring-ai-openai-spring-boot-starter</artifactId>
    </dependency>
  </dependencies>
</project>
```

application.yml — Configuration

```
spring:
  application:
    name: spring-ai-demo

  ai:
    openai:
      api-key: ${OPENAI_API_KEY}
      chat:
        options:
          model: gpt-4o-mini
          temperature: 0.7
          max-tokens: 1000

server:
  port: 8080
```

4.5 Writing Your First Spring AI Chat Application

With the project configured, let's build a complete REST API that exposes AI chat functionality.

ChatService.java

```
package com.agenticai.service;

import org.springframework.ai.chat.client.ChatClient;
import org.springframework.ai.chat.model.ChatModel;
import org.springframework.ai.chat.prompt.Prompt;
import org.springframework.ai.chat.prompt.SystemPromptTemplate;
import org.springframework.ai.chat.messages.UserMessage;
import org.springframework.stereotype.Service;

@Service
public class ChatService {

    private final ChatClient chatClient;

    public ChatService(ChatModel chatModel) {
        this.chatClient = ChatClient.builder(chatModel)
            .defaultSystem("You are a helpful AI assistant specialized in " +
                           "Java and Agentic AI development.")
            .build();
```

```
    }

    public String ask(String question) {
        return chatClient
            .prompt()
            .user(question)
            .call()
            .content();
    }

    public String askWithContext(String systemContext, String question) {
        return chatClient
            .prompt()
            .system(systemContext)
            .user(question)
            .call()
            .content();
    }
}
```

ChatController.java

```
package com.agenticai.controller;

import com.agenticai.service.ChatService;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/chat")
public class ChatController {

    private final ChatService chatService;

    public ChatController(ChatService chatService) {
        this.chatService = chatService;
    }

    @PostMapping
    public ChatResponse chat(@RequestBody ChatRequest request) {
        String response = chatService.ask(request.message());
        return new ChatResponse(response);
    }

    @PostMapping("/contextual")
    public ChatResponse contextualChat(@RequestBody ContextualChatRequest request)
{
        String response = chatService.askWithContext(
            request.systemContext(), request.message()
        );
        return new ChatResponse(response);
    }

    record ChatRequest(String message) {}
```

```
    record ContextualChatRequest(String systemContext, String message) {}
    record ChatResponse(String reply) {}
}
```

Streaming Response Controller

```
import reactor.core.publisher.Flux;
import org.springframework.http.MediaType;

@GetMapping(value = "/stream", produces = MediaType.TEXT_EVENT_STREAM_VALUE)
public Flux<String> streamChat(@RequestParam String message) {
    return chatClient
        .prompt()
        .user(message)
        .stream()
        .content();
}
```

Structured Output — Getting Typed Java Objects

```
record MovieRecommendation(String title, String genre, int year, String reason) {}

MovieRecommendation movie = chatClient
    .prompt()
    .user("Recommend one sci-fi movie for a Java developer. Respond with JSON.")
    .call()
    .entity(MovieRecommendation.class);

System.out.println("Title: " + movie.title());
System.out.println("Year: " + movie.year());
```

Testing Your Endpoints with curl

```
# Start the application
mvn spring-boot:run

# Test basic chat
curl -X POST http://localhost:8080/api/chat \
  -H "Content-Type: application/json" \
  -d '{"message": "What is Spring AI in one sentence?"}'

# Test streaming
curl http://localhost:8080/api/chat/stream?message=Explain+context+windows
```

Pro Tip

Create your Spring Boot project via start.spring.io, import into IntelliJ IDEA, and run it before writing any AI code. A basic /health endpoint should return 200 OK. Only then add AI dependencies — this isolates environment problems from AI-specific issues.

4.6 ChatClient vs ChatModel — When to Use Each

Aspect	ChatModel (Low-level)	ChatClient (High-level)
API Style	Direct call with Prompt object	Fluent builder pattern
System Prompt	Manual message construction	.defaultSystem() or .system() per call
Memory	Must manage manually	Advisors handle automatically
RAG	Must implement manually	QuestionAnswerAdvisor handles it
Streaming	Direct Flux access	.stream() method
Best For	Custom low-level control	Most application use cases

4.7 PromptTemplate — Dynamic Prompts

Spring AI provides PromptTemplate for creating reusable, parameterized prompts — similar to string templates but with first-class AI tooling support.

```
String template = """
    You are analyzing Java code quality.

    Code to review:
    {code}

    Focus areas: {focusAreas}

    Provide a structured review with: issues found, severity (HIGH/MEDIUM/LOW), and
suggested fixes.
    """;

PromptTemplate promptTemplate = new PromptTemplate(template);
Prompt prompt = promptTemplate.create(Map.of(
    "code", sourceCode,
    "focusAreas", "performance, null safety, exception handling"
));

String review = chatModel.call(prompt).getResult().getOutput().getContent();
```

Key Terms — Session 4 Quick Reference

Term	Definition
Spring AI BOM	Bill of Materials POM that manages all Spring AI dependency versions consistently.
ChatClient	High-level fluent API for interacting with any LLM in Spring AI applications.

ChatModel	Lower-level interface; direct prompt-response API for a specific provider.
Advisor	Middleware for ChatClient — implements cross-cutting concerns like memory and RAG.
Auto-Configuration	Spring Boot mechanism that auto-creates beans from classpath and application.yml.
Structured Output	Spring AI feature to parse LLM response directly into typed Java POJOs.
PromptTemplate	Template class for creating reusable prompts with variable substitution.
Provider Portability	Spring AI's ability to swap between LLM providers with only config changes.

Session 1	LLM Theory: Open/closed source models, tokenization, context windows, sampling strategies (temperature/top-p/top-k), hallucination causes and mitigations, cost and latency trade-offs.
Session 2	Java LLM Integration: API vs SDK vs Framework decision, REST API fundamentals, OpenAI Java SDK, first working Java AI application with multi-turn conversation support.
Session 3	Hugging Face: Platform overview, 800K+ model hub, Inference API, authentication with HF tokens, Java REST integration for NLP tasks (sentiment, generation, embeddings), Spring AI HF provider.
Session 4	Spring AI: Architecture, provider portability, auto-configuration, full Spring Boot project setup (pom.xml + application.yml), ChatModel vs ChatClient, streaming, structured output, PromptTemplate.

End-of-Day Deliverables

- A working ConversationService in Java that maintains multi-turn chat history using the OpenAI SDK.
- A HuggingFaceClient that can call at least three different task types (sentiment, generation, embeddings).
- A Spring Boot project with pom.xml, application.yml, ChatService, and ChatController exposing /api/chat.
- Understanding of when to use temperature=0 vs 0.7 and why this matters for agentic pipelines.